ROS2 Humble 설치 & 환경설정

설치 · 환경설정 · 워크스페이스

cchyun@gmail.com

Part 1. ROS2 필요성과 아키텍처 개요

ROS가 없던 시절의 로봇 개발 — "왜 ROS인가?"

- **로봇 개발 팀 '로보파이오니어'의 협업 시나리오**
  - **철수(하드웨어):** 라이다(LiDAR) 센서 데이터를 읽는 C 코드
  - **영희(알고리즘):** 파이썬으로 장애물 회피 알고리즘
  - **민수(제어):** 로봇 바퀴를 움직이는 C++ 모터 제어 코드
- **ROS가 없을 때 발생하는 구체적인 문제들**
  - **통신 지옥 (Protocol Design)**
    - 철수 C 코드 → 영희 파이썬: 전송 속도·데이터 포맷·소켓 통신 규약을 처음부터 정의
    - 영희가 알고리즘 수정할 때마다 철수·민수도 통신 코드 수정 반복
  - **의존성 파편화 (Single Executable)**
    - 모든 기능을 하나의 거대한 코드 파일에 집어넣으면, 오타 하나에 로봇 전체 멈춤
  - **바퀴의 재발명 (No Reusability)**
    - 센서 데이터 시각화, 상태 그래프 도구도 직접 GUI 코딩해야 함

ROS2란 무엇인가?

- **ROS2는 로봇용 미들웨어(운영체제)**
  - 복잡한 통신 인프라와 도구를 미리 만들어 제공
  - 개발자가 **'로직'에만 집중**할 수 있게 해줌
- **로보파이오니어 팀의 결과**
  - 철수의 C 코드 라이다 데이터 + 영희의 파이썬 알고리즘
  - `rclpy`와 `topic`을 통해 **단 몇 줄의 코드**로 연결
  - `colcon build`로 안정적인 시스템 구축

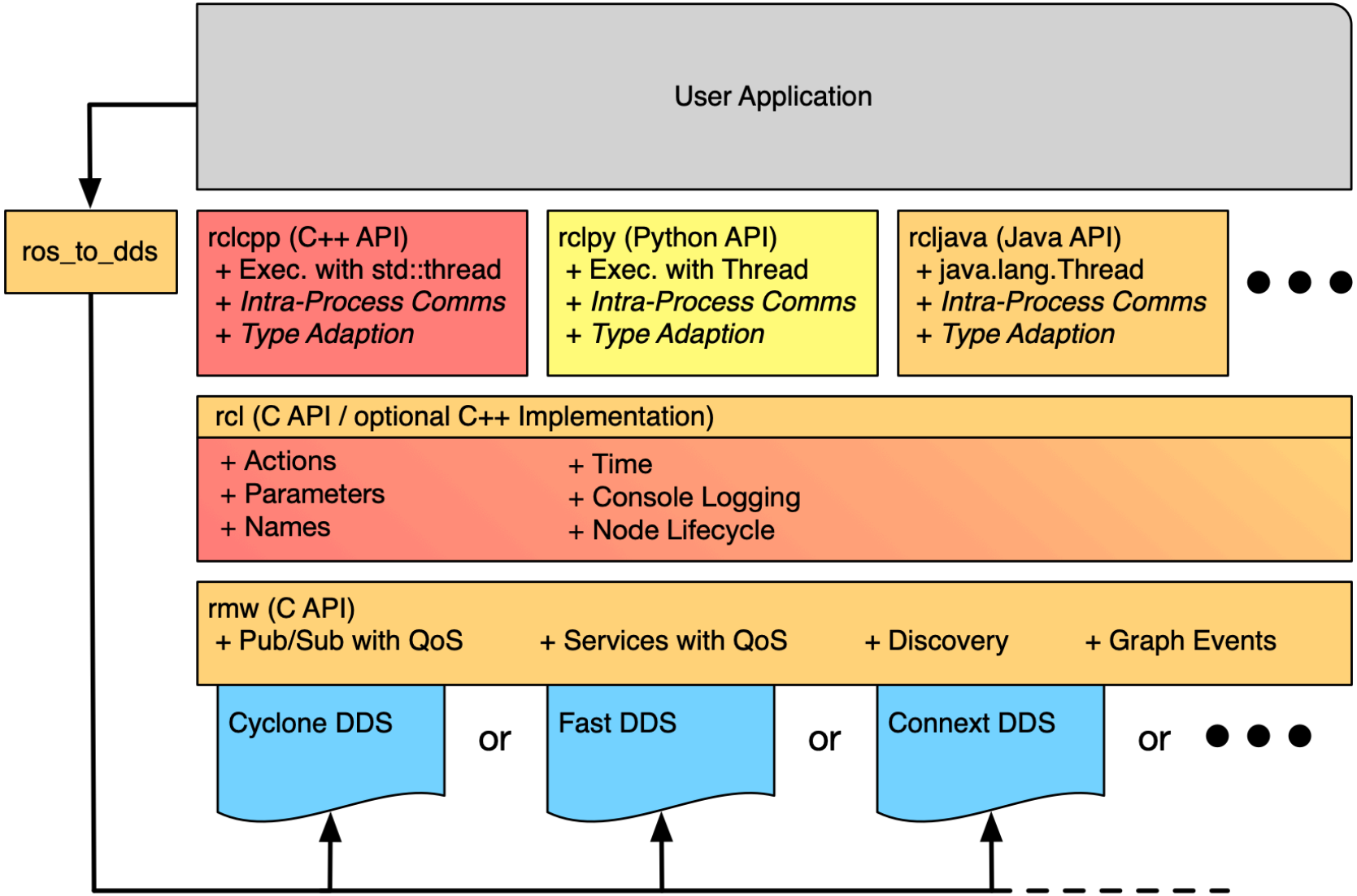
ROS 1 → ROS 2로: 핵심 개선 이유

- **왜 구버전(ROS 1) 대신 ROS 2를 선택해야 할까?**
  - **실시간성 (Real-time)**
    - **QoS(Quality of Service)** 설정 도입
    - "모터 제어 명령은 절대 놓치면 안 됨(Reliable)" vs "센서 데이터는 최신 것만 중요함(Best-effort)"
  - **DDS 통신 (Decentralization)**
    - ROS 1: Master 노드가 죽으면 시스템 전체 멈춤
    - ROS 2: **DDS(Data Distribution Service)** 로 노드들이 서로를 직접 찾는 분산형 구조
  - **보안 (Security)**
    - DDS 수준의 데이터 암호화 및 인증 기능
  - **멀티 플랫폼 지원**
    - 리눅스(Ubuntu)뿐만 아니라 **Windows, macOS** 공식 지원

ROS 2 전체 아키텍처 — "DDS라는 튼튼한 다리"

- **ROS2의 계층 구조**
  - **사용자 코드 층** — 여러분이 짤 파이썬 코드
  - **클라이언트 라이브러리 (rclpy / rclcpp)** — 파이썬·C++로 ROS2 기능을 쉽게 쓰게 해주는 API
  - **RCL (ROS Client Library)** — 언어 무관 공통 로직(노드 관리, 통신 규칙)이 구현된 C 기반 핵심 계층
  - **RMW (ROS Middleware)** — DDS와 ROS를 연결해주는 브리지
  - **DDS (Data Distribution Service)** — 최하단에서 실제 데이터를 실어 나르는 도로
- **DDS가 로봇에 적합한 이유**
  - 산업 표준 통신 규격, 중앙 서버 없이 **"이 데이터를 원하는 사람?"** 하고 바로 연결(Discovery)
  - 네트워크가 불안정한 무선 환경에서도 안정적인 데이터 송수신
  - 이동하며 일하는 로봇에 최적화된 구조

ROS 2 전체 아키텍처



* *Intra-Process Comms* and *Type Adaption* could be implemented in the client library, but may not currently exist.

ROS2 배포판과 Humble을 선택하는 이유

- **ROS2에도 윈도우 10, 11처럼 배포판(Distribution)이 있음**
  - **LTS (Long-Term Support)**
    - 장기 지원 버전: 한 번 설치하면 오랫동안 업데이트·보안 지원 제공
  - **Humble Hawksbill (Humble)**
    - 널리 쓰이는 LTS 배포판
  - **Ubuntu 22.04 대응**
    - 대학교와 산업계 표준 Ubuntu 22.04와 찰떡궁합
    - 안정적인 개발 환경 보장

빌드 도구와 라이브러리 — "누가 무엇을 하는가?"

- **로봇 팀이 프로젝트를 빌드하고 코드를 짤 때 쓰는 핵심 구성 요소**
  - **ament** — 빌드 시스템 (레시피 역할)
    - ROS2 패키지를 어떻게 구성하고 빌드할지 정의
  - **colcon** — 빌드 도구 (요리사 역할)
    - 여러 개의 패키지를 한꺼번에 빌드
  - **rclpy** — ROS Client Library for Python
    - 파이썬 입문자가 가장 많이 쓰게 될 라이브러리
  - **rclcpp** — C++용 클라이언트 라이브러리
    - 성능이 중요한 모터 제어 등에 사용
- **관계:** `rclpy`로 코드 작성 → `ament`에 맞게 패키지 구성 → `colcon build`로 빌드

Part 2. ROS2 Humble 설치 전체 절차

설치 전 — Locale 설정 & GPG 키 추가

- **Locale 설정: UTF-8을 지원하도록 로케일 설정**

```
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

- **GPG 키 추가: ROS2 저장소 패키지가 신뢰할 수 있는지 확인하기 위한 보안 키 등록**

```
sudo apt update && sudo apt install curl gnupg lsb-release
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg
```

apt 소스 등록 & ROS2 Humble 설치

- **apt 소스 등록: 패키지 매니저가 ROS2 패키지를 찾을 수 있도록 저장소 목록에 추가**

```
echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    http://packages.ros.org/ros2/ubuntu \
    $(source /etc/os-release && echo $UBUNTU_CODENAME) main" | \
    sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

- **ROS2 Humble 설치: 저장소 업데이트 후 데스크탑 버전 설치**

```
sudo apt update
sudo apt install ros-humble-desktop    # GUI 도구를 포함한 전체 설치
```

rosdep 초기화

- **rosdep: 의존성 관리 도구 — 패키지 빌드 시 필요한 도구를 자동 설치**

```
sudo apt install python3-rosdep
sudo rosdep init
rosdep update
```

- **예상 출력:**

```
Reading targets from ...
Hit ...
```

→ 성공적으로 데이터베이스가 업데이트되었다는 메시지 출력

colcon 설치

- colcon: ROS2 패키지를 빌드하는 공식 빌드 도구

```
sudo apt install python3-colcon-common-extensions
```

- 설치 확인

```
colcon version-check
```

Desktop vs ROS-Base 설치 선택 기준

- **ROS2는 다양한 설치 변체(Variant)를 제공**

  - **ROS-Desktop (추천)**
    - ROS2 핵심 라이브러리
    - **RViz2** — 시각화 도구
    - **RQt** — GUI 도구
    - 시뮬레이션 관련 데모 패키지
    - **로봇 개발 및 학습용 PC에 적합**

  - **ROS-Base**
    - GUI 없는 최소 설치 버전
    - 통신 라이브러리와 핵심 CLI 도구만 포함
    - **하드웨어 성능이 제한적인 실제 로봇 내부 컴퓨터·임베디드 장치에 적합**

Part 3. 환경설정 & 워크스페이스

source setup.bash의 의미와 영구 등록

- `source /opt/ros/humble/setup.bash` 의 의미
  - ROS2 시스템의 실행 파일, 라이브러리, 환경 변수(Path 등)를 현재 터미널 세션에 로드
  - 매번 새 터미널을 열 때마다 입력하지 않으면 ROS2 명령어 인식 불가

- **영구 등록: `.bashrc` 파일 하단에 추가**

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

- **환경변수 확인 방법**

```
printenv | grep ROS
```

```
ROS_DISTRO=humble
ROS_VERSION=2
ROS_PYTHON_VERSION=3
```

ROS2 Workspace 생성 및 구조 설명

- **ROS2 프로젝트는 Workspace라는 구조화된 폴더 내에서 관리**
  - `~/Workspace/ros2_ws/src/` — 사용자가 직접 작성하거나 외부에서 가져온 소스 코드(Packages)
  - `build/` — `colcon build` 실행 시 생성되는 중간 파일(CMake 캐시, 오브젝트 파일 등)
  - `install/` — 빌드 완료된 최종 실행 파일과 라이브러리; 실행 시 이 폴더 설정을 참조
  - `log/` — 빌드 과정에서 발생한 상세 로그 기록

- **워크스페이스 생성 명령어**

```
mkdir -p ~/Workspace/ros2_ws/src
cd ~/Workspace/ros2_ws
colcon build
```

colcon build 후 source install/setup.bash가 필요한 이유

- `colcon build` **명령어 실행 흐름**
  - 패키지를 빌드하면 실행 파일이 `install/` 폴더 내에 생성
  - **운영체제는 이 새로운 패키지가 어디 있는지 알지 못함**

- `source install/setup.bash` **역할**
  - 현재 워크스페이스에서 빌드된 패키지들을 시스템 경로에 추가
  - `ros2 run`이나 `ros2 launch` 명령어로 사용할 수 있게 됨

- **권장 순서**

```
cd ~/Workspace/ros2_ws
colcon build
source install/setup.bash
ros2 run <패키지명> <노드명>
```

Part 4. 자주 발생하는 오류와 해결법

설치 오류 해결 — GPG 키·rosdep 문제

- **GPG 키 오류**
  - 키 추가 과정에서 네트워크 문제로 키가 누락된 경우
  - 키를 삭제하고 다시 수동으로 다운로드하여 등록

- **rosdep 초기화 실패**
  - `sudo rosdep init` 실행 시 "이미 파일이 존재한다"는 에러 발생 시:

```
sudo rm /etc/ros/rosdep/sources.list.d/20-default.list
sudo rosdep init
rosdep update
```

  - 네트워크 오류(SSL 관련) 발생 시: 일시적인 서버 문제일 수 있음
    - 잠시 후 재시도하거나 테더링 등으로 네트워크 환경 변경

빌드·통신 오류 해결

- **colcon build 실패 — 주로 의존성 패키지 부족**
  - 빌드 전 누락된 의존성을 자동 설치:

```
rosdep install -i --from-path src --rosdistro humble -y
colcon build
```

- **패키지를 찾을 수 없다는 오류 — 대부분 Sourcing 문제**
  - 다음 두 가지가 모두 실행되었는지 확인:

```
source /opt/ros/humble/setup.bash        # 각 터미널마다 필요
source ~/ros2_ws/install/setup.bash      # 빌드 후 항상 실행
```

입문자 "멘붕" 포인트 3가지

- **① "QoS가 안 맞아서 데이터가 안 보여요!" (QoS Mismatch)**
  - 한쪽은 'Reliable', 받는 쪽은 'Best-effort' → 통신 연결 자체가 안 됨
  - **해결:** `rqt_graph`로 노드 연결 확인 후 통신 방식 일치

- **② "파이썬 패키지인데 왜 CMake를 써야 하죠?" (Interface Package)**
  - 커스텀 메시지(.msg) 파일 생성 시 빌드 형식을 `ament_cmake`로 설정해야 함
  - **해결:** 인터페이스(데이터 형식)만 정의하는 전용 패키지를 따로 만드는 것이 ROS2 관례

- **③ "setup.py와 package.xml이 헷갈려요!"**
  - 패키지 정보를 두 군데 다 적어야 해서 혼란스러움
  - **해결:**
    - `package.xml` → ROS 시스템 전체가 내 패키지를 이해하기 위한 **명세서**
    - `setup.py` → 파이썬 도구들이 내 코드를 설치하기 위한 **설치 설명서**

Part 5. ROS2 동작 확인 — turtlesim 실습

turtlesim이란?

- **ROS2 설치 후 동작을 확인하는 가장 기본적인 실습 패키지**
  - 거북이(turtle)를 화면에 띄우고 키보드로 움직이는 간단한 시뮬레이터
  - 별도 하드웨어 없이 **노드·토픽·서비스의 동작을 한눈에 확인** 가능
  - ROS2 설치 시 기본 포함 (`ros-humble-desktop` 기준)
- **이번 실습에서 확인할 것**
  - 노드가 정상적으로 실행되는가?
  - 토픽이 발행·구독되는가?
  - 키보드 입력이 `geometry_msgs/msg/Twist` 메시지로 전달되는가?

turtlesim 실행 — 터미널 구성

- **터미널 3개를 준비하여 각각 역할 분리**

터미널	역할	실행 명령어
터미널 1	turtlesim 노드 실행	`ros2 run turtlesim turtlesim_node`
터미널 2	키보드 조작 노드 실행	`ros2 run turtlesim turtle_teleop_key`

- **터미널마다 아래 명령어가 실행된 상태여야 함**

```
source /opt/ros/humble/setup.bash
```

turtlesim 실행 — Step 1 & 2

- **[터미널 1] turtlesim 노드 실행**

```
ros2 run turtlesim turtlesim_node
```

```
# 출력 예시
[INFO] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [turtlesim]: Spawning turtle [turtle1] at x=[5.54], y=[5.54], theta=[0.00]
```

→ 파란 배경에 거북이가 표시되는 창이 열리면 정상

turtlesim 실행 — Step 1 & 2

- **[터미널 2] 키보드 조작 노드 실행**

```
ros2 run turtlesim turtle_teleop_key
```

```
# 출력 예시
Reading from keyboard
---------------------------
Use arrow keys to move the turtle.
```

→ **터미널 2에 포커스를 두고** 방향키로 거북이 조작